

AI Gateway 项目规划

一、项目概述

项目名称

二、项目背景

行业变化

企业真正遇到的问题

三、系统架构

整体产品定位

部署架构

整个系统架构

四、产品功能

首页 (Dashboard)

用户管理 (Users)

组织管理 (Organizations)

认证中心 (Authentication)

API Keys

供应商 Provider

模型 (Models)

智能路由 (Router)

用量分析 (Usage)

额度配置 (Quotas)

Logs (日志)

Cost (成本)

Announcement (公告)

Billing (计费)

System Settings (系统设置)

Audit Log (审计日志)

策略引擎 (Policy Engine)

一句话理解

Cost Engine 是什么？

Cost Engine 页面应该配置什么？

Router 和 Cost Engine 的关系

Cost Engine 总体架构

- ① 成本中心 (Cost Center)
- ② 定价中心 (Pricing)
- ③ 利润分析 (Profit Analytics)
- ④ 成本优化 (Optimization)
- ⑤ Budget (预算管理)
- ⑥ Alerts (成本预警)
- ⑦ 策略中心 (Policies)
- ⑧ 报表中心 (Reports)

ROI 分析

模型推荐

Cost Engine 页面结构建议

MVP 阶段哪些必须做？

增加 Policy Engine 请求时间处理方案

五、Roadmap

一、项目概述

项目名称

Nova AI Gateway

一句话介绍：面向企业和 AI 开发者的大模型统一接入平台，提供模型聚合、智能路由、成本优化、Agent Runtime 以及行业 AI 能力。

二、项目背景

行业变化

2023 年 AI 公司："我要接 GPT"

2024 年 AI 公司："我要接 GPT + Claude"

2025 年 AI 公司："我要接十几个模型"

2026 年 AI 公司："我要根据成本、速度、效果自动选择模型"

整个 AI 行业已经进入：**Multi-Model（多模型时代）**

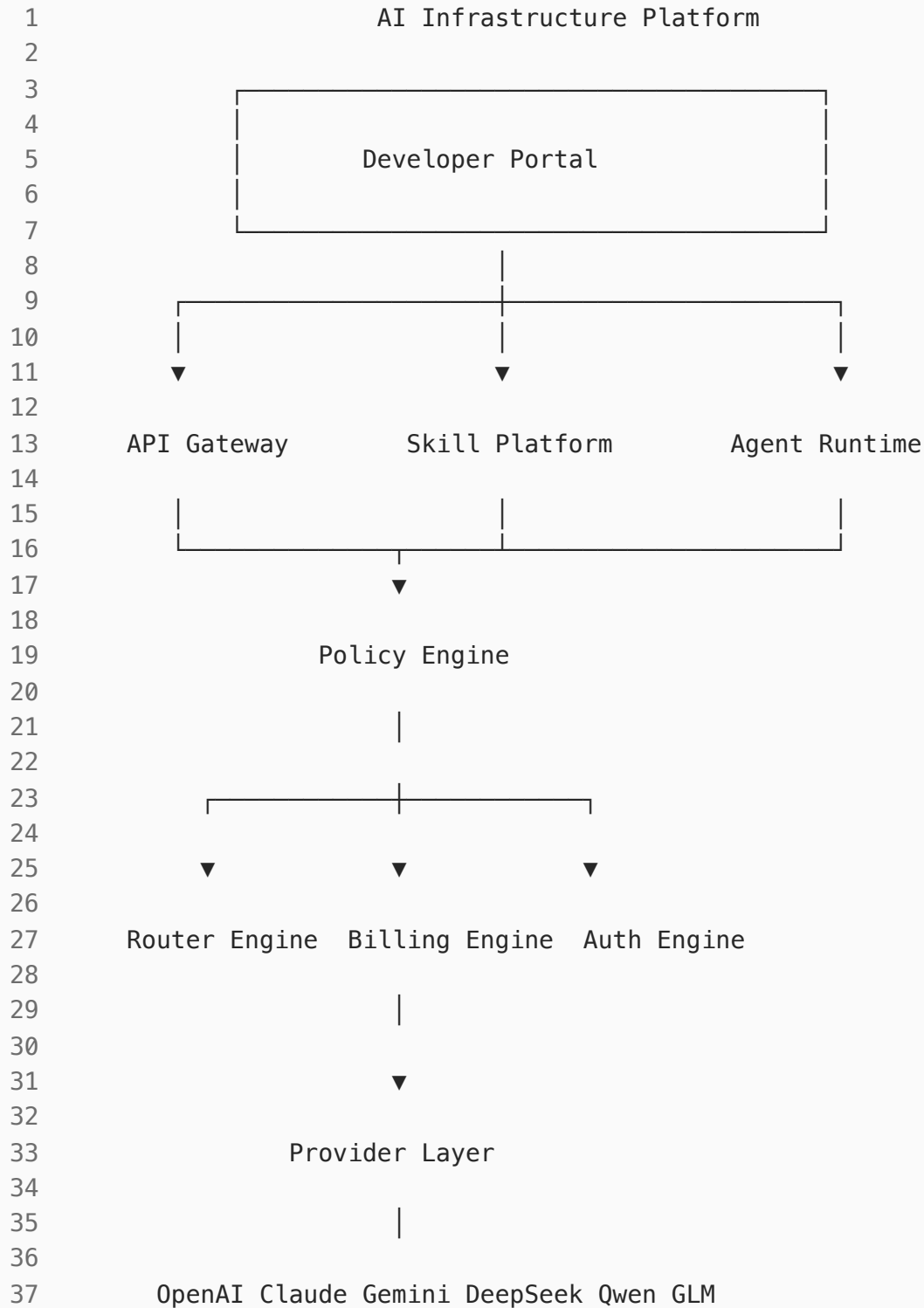
因此：企业需要的不再是：GPT API，而是：AI Infrastructure。

企业真正遇到的问题

问题	描述
模型越来越多	OpenAI、Claude、Gemini、Qwen、DeepSeek...
API 不统一	参数、返回值、能力各不相同
成本越来越高	不同模型价格差异巨大
稳定性不足	单一 Provider 故障影响业务
权限混乱	多团队、多项目难管理
日志缺失	无法追踪 Token 消耗与问题定位

三、系统架构

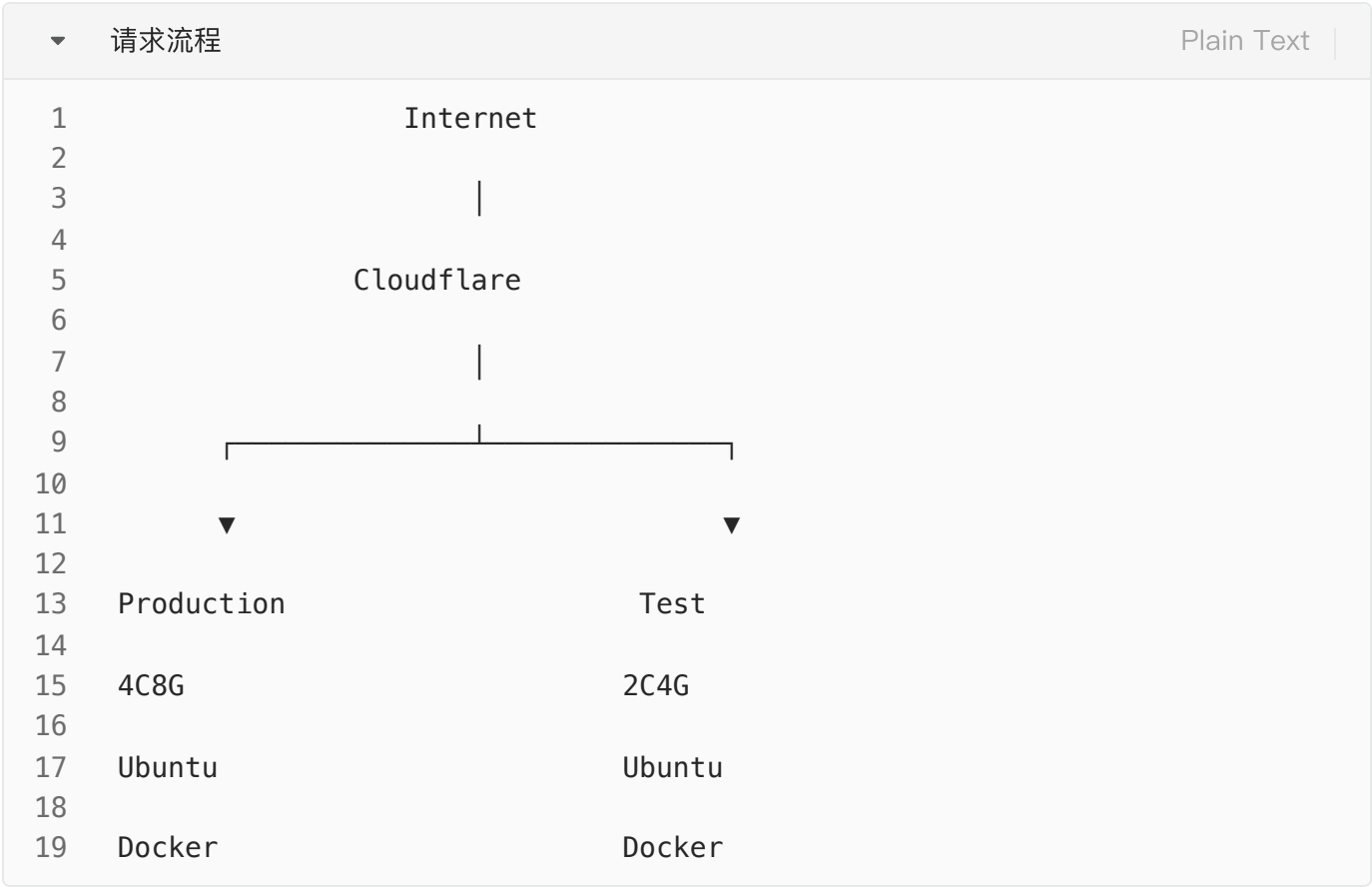
整体产品定位



部署架构

✅ 新加坡生产服务器

✅ 新加坡测试服务器



生产服务器里面：

```
1  Docker
2
3  └─ Nginx
4
5  └─ Gateway
6
7  └─ Policy Engine
8
9  └─ Router Engine
10
11 └─ Auth Service
12
13 └─ Billing Service
14
15 └─ PostgreSQL
16
17 └─ Redis
18
19 └─ MinIO (可选)
```

全部：

Docker Compose。

第一阶段：

不要K8S

整个系统架构

建议拆：

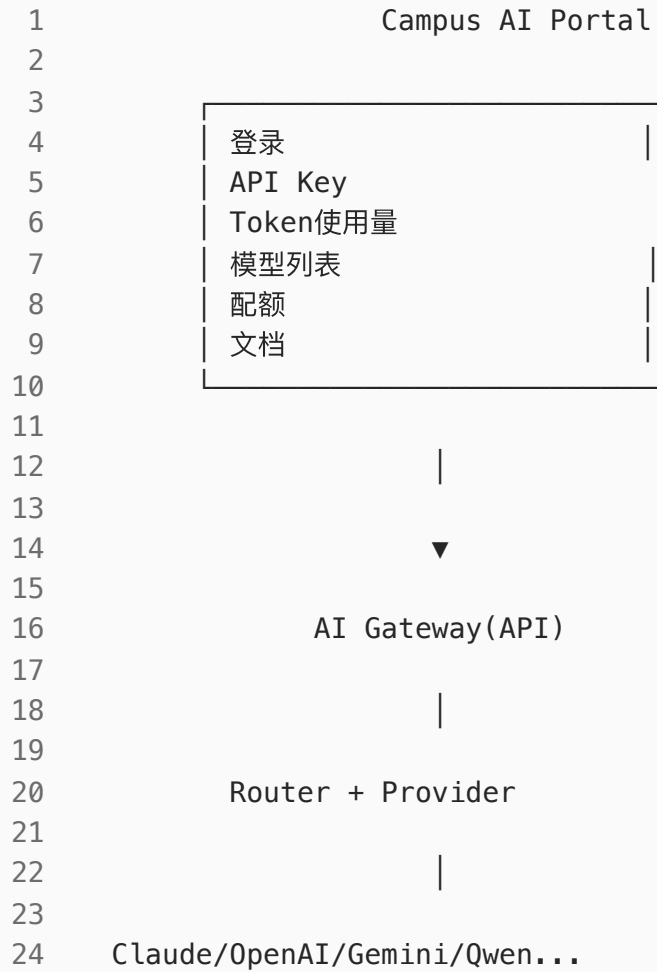
```
1  AI Platform
2
3  |
4
5  ├── Web Console
6
7  ├── Admin Console
8
9  ├── API Gateway
10
11 ├── Policy Engine
12
13 ├── Router Engine
14
15 ├── Provider Adapter
16
17 ├── Billing Engine
18
19 ├── Event Bus
20
21 ├── Cost Engine
22
23 ├── Usage Engine
24
25 ├── Log Engine
26
27 ├── Notification
28
29 └── MCP Gateway (以后)
```

四、产品功能

提供 OpenAI Compatible API + 校园账号体系 + 配额管理。

Portal 只是管理后台。

真正的核心产品是：



首页 (Dashboard)

这是运营每天都会看的页面

作用： 整个AI平台运行情况

展示

今日		Plain Text
1	今日请求数	
2		
3	今日Token	
4		
5	今日收入	
6		
7	今日成本	
8		
9	今日利润	

实时		Plain Text
1	在线用户	
2		
3	活跃API Key	
4		
5	当前QPS	
6		
7	平均延迟	
8		
9	成功率	
10		
11	失败率	

模型排行榜		Plain Text
1	GPT	
2		
3	Claude	
4		
5	DeepSeek	
6		
7	Gemini	
8		
9	Qwen	

用户管理（Users）

管什么？

所有注册用户

页面字段

▼ 页面字段			Plain Text
1	字段	说明	
2	-----	-----	
3	User ID	唯一ID	
4	昵称	用户名称	
5	邮箱	登录账号	
6	组织	所属组织	
7	套餐	Free / Pro / Enterprise	
8	余额	账户余额	
9	API数量	拥有多少Key	
10	请求次数	累计请求	
11	状态	正常/禁用	
12			

操作

管理员可以：

- 创建用户
- 禁用用户
- 重置密码
- 调整余额
- 修改套餐
- 查看调用记录

组织管理（Organizations）

如果以后支持企业

例如

▼		Plain Text
1	OpenAI中国	
2		
3	腾讯	
4		
5	比亚迪	
6		
7	星诺创新	

每家公司：

都有：

▼		Plain Text
1	成员	
2		
3	额度	
4		
5	预算	
6		
7	权限	

管理员：

可以：

▼		Plain Text
1	新增组织	
2		
3	删除组织	
4		
5	邀请成员	
6		
7	设置管理员	

认证中心（Authentication）

支持：

- 学校邮箱注册
- Google 登录（如果有国际高校）
- GitHub 登录（计算机专业学生）
- 邮箱验证码登录

后续：

CAS（校园统一认证）

SSO

再接

API Keys

每个用户：可以，创建多个 Key

例如：

▼	创建API key	Plain Text
1	sk-campus-xxxxx	

字段：

▼		Plain Text
1	字段	说明
2	-----	-----
3	Key名称	开发环境
4	Key前缀	sk-xxxx
5	所属用户	谁的
6	权限	只读/全部
7	状态	启用
8	创建时间	什么时候
9	最近调用	多久前
10		

点击：

进入详情：

可以看到：

▼		Plain Text
1	Token	
2		
3	花费	
4		
5	最近100次请求	
6		
7	IP地址	
8		
9	错误率	

支持：

- 创建
- 删除
- 重置

供应商 Provider

后台：

配置：

▼ Provider		Plain Text
1	Claude	
2		
3	OpenAI	
4		
5	Gemini	
6		
7	DeepSeek	
8		
9	Qwen	

每个Provider：

配置：

▼ Plain Text	
1	Base URL
2	
3	API Key
4	
5	Timeout
6	
7	Weight
8	
9	Priority
10	
11	QPS
12	
13	Retry
14	
15	Enable

例如：

▼ 配置示例 Plain Text	
1	OpenAI
2	
3	https://api.openai.com
4	
5	Key
6	
7	sk-xxx
8	
9	Weight
10	
11	100

模型（Models）

不要和Provider混。

例如：

GPT5：

可能：

来自：

▼ Plain Text	
1	OpenAI
2	
3	Azure
4	
5	OpenRouter

所以：

Model：

是：

逻辑模型。

Provider：

是真实供应商。

▼ Plain Text	
1	GPT5
2	
3	Claude Sonnet
4	
5	Gemini Pro
6	
7	DeepSeek R1
8	
9	Qwen Max

每个：

配置：

		▼	Plain Text
1	价格		
2			
3	支持图片		
4			
5	支持Function		
6			
7	支持Streaming		
8			
9	最大Context		
10			
11	是否启用		

智能路由（Router）

这是：Gateway核心

例如：

		▼	Router	Plain Text
1	GPT5			
2				
3	↓			
4				
5	OpenAI			
6				
7	Weight 80			
8				
9	↓			
10				
11	Azure			
12				
13	Weight 20			

或者：

▼		Plain Text
1	Claude	
2		
3	↓	
4		
5	Anthropic	
6		
7	↓	
8		
9	OpenRouter	
10		
11	↓	
12		
13	AWS Bedrock	

策略：

▼		Plain Text
1	最低价格	
2		
3	最快速度	
4		
5	权重	
6		
7	轮询	
8		
9	失败切换	

用量分析（Usage）

统计，例如：

今日：

▼ Usage		Plain Text
1	Claude	
2		
3	120万Token	
4		
5	GPT	
6		
7	80万Token	

以及：费用

额度配置 （Quotas）

管理：

用户：

一天：

可以：

▼		Plain Text
1	Token	
2		
3	Requests	
4		
5	Images	
6		
7	Audio	

例如：

Free：

▼		Plain Text
1	100次/天	

Pro：

▼		Plain Text
1	10000次	

Enterprise:

无限。

还能:

限制:

▼		Plain Text
1	GPT	
2		
3	只能100次	
4		
5	Claude	
6		
7	500次	

Logs（日志）

每一次请求:

都在这里。

例如:

▼		Plain Text
1	时间	
2		
3	用户	
4		
5	API Key	
6		
7	模型	
8		
9	Provider	
10		
11	Prompt长度	
12		
13	Completion长度	
14		
15	耗时	
16		
17	IP	
18		
19	状态	
20		
21	错误	

还能：

查看：

Request：

Response。

方便排查。

Cost（成本）

这是商业核心。

很多平台都没有。

建议做。

例如：

▼	Plain Text
1	OpenAI
2	
3	收入:
4	
5	100
6	
7	成本:
8	
9	70
10	
11	利润:
12	
13	30

还能:

按:

▼	Plain Text
1	用户
2	
3	模型
4	
5	Provider

统计。

例如:

▼	Plain Text
1	DeepSeek
2	
3	利润率:
4	
5	90%

GPT:

▼ Plain Text	
1	利润：
2	
3	20%

以后：

Router：

自动：

优先：

利润高。

Announcement（公告）

运营：

发：

▼ Plain Text	
1	系统升级
2	
3	模型更新
4	
5	维护通知
6	
7	新模型上线

用户：

登录：

就看到。

Billing（计费）

管理：

▼		Plain Text
1	套餐	
2		
3	订单	
4		
5	充值	
6		
7	退款	
8		
9	优惠券	
10		
11	发票	

如果商业化，这个几乎是必备模块。

System Settings（系统设置）

统一管理平台配置：

- 默认模型
- Token 汇率/价格
- 默认超时时间
- 默认重试次数
- SMTP 邮件配置
- 第三方登录配置
- Webhook
- API 限流参数

Audit Log（审计日志）

记录所有后台操作：

1 管理员A
2
3 修改了
4
5 OpenAI Provider
6
7 ↓
8
9 从:
10
11 Weight=100
12
13 改成:
14
15 Weight=20

策略引擎 (Policy Engine)

一句话理解

```

1  用户请求
2
3      |
4
5      ▼
6
7  Cost Engine (决策中心)
8
9      |
10
11 告诉 Router:
12
13  "这次走 Claude"
14
15  "这次走 DeepSeek"
16
17  "这次走 GPT"
18
19      |
20
21      ▼
22
23  Router
24
25  真正把请求发送出去
    
```

Cost Engine 是什么？

Cost Engine：

就是：

AI平台的大脑

它负责：

分析：

▼	Plain Text
1	这个请求：
2	
3	应该赚多少钱？
4	
5	应该走哪个模型？
6	
7	应该收费多少？
8	
9	利润是多少？

例如：

现在：

▼	Plain Text
1	GPT5
2	
3	成本：
4	
5	¥0.12
6	
7	收费：
8	
9	¥0.15
10	
11	利润：
12	
13	¥0.03

DeepSeek：

		▼	Plain Text
1	成本：		
2			
3	¥0.02		
4			
5	收费：		
6			
7	¥0.15		
8			
9	利润：		
10			
11	¥0.13		

Cost Engine：

就会判断：

		▼	Plain Text
1	如果用户没指定GPT		
2			
3	↓		
4			
5	走DeepSeek		

Cost Engine页面应该配置什么？

例如：

▼ Plain Text	
1	DeepSeek
2	
3	采购：
4	
5	¥2
6	
7	售价：
8	
9	¥4
10	
11	利润：
12	
13	¥2

Claude：

▼ Plain Text	
1	采购：
2	
3	¥10
4	
5	售价：
6	
7	¥15

GPT：

▼ Plain Text	
1	采购：
2	
3	¥15
4	
5	售价：
6	
7	¥20

Cost Engine：

计算：

▼ Plain Text	
1	利润率
2	
3	毛利
4	
5	ROI
6	
7	每用户利润
8	
9	每组织利润
10	
11	每模型利润

还能：

配置：

例如：

▼ Plain Text	
1	目标：
2	
3	利润率：
4	
5	40%

系统：

自动：

调整：

售价。

更高级一点

以后：

Cost Engine：

甚至可以：

自动：

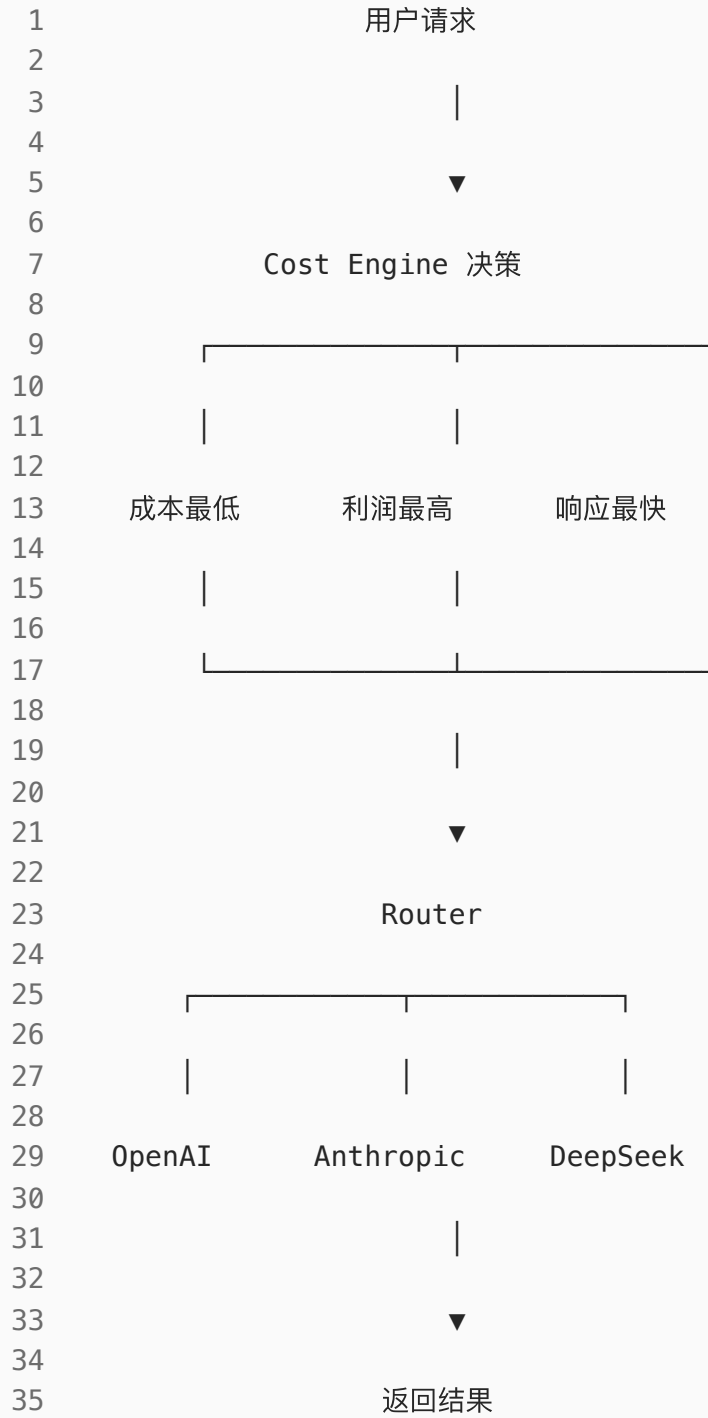
选择：

	▼	Plain Text
1	利润最高	
2		
3	↓	
4		
5	优先	

例如：

	▼	Plain Text
1	客户：	
2		
3	普通聊天	
4		
5	↓	
6		
7	GPT：	
8		
9	利润：	
10		
11	10%	
12		
13	Claude：	
14		
15	利润：	
16		
17	30%	
18		
19	DeepSeek：	
20		
21	利润：	
22		
23	80%	

Router 和 Cost Engine 的关系



例如：

▼		Plain Text
1	企业VIP用户	
2		
3	↓	
4		
5	允许使用GPT-5	
6		
7	↓	
8		
9	预算充足	
10		
11	↓	
12		
13	优先最快Provider	

而：

▼		Plain Text
1	免费用户	
2		
3	↓	
4		
5	预算有限	
6		
7	↓	
8		
9	优先DeepSeek	
10		
11	↓	
12		
13	每天最多100次	

Cost Engine 总体架构

▼ Cost Engine		Plain Text
1	Cost Engine	
2		
3	├─ 成本中心 (Cost Center)	
4	├─ 定价中心 (Pricing)	
5	├─ 利润分析 (Profit Analytics)	
6	├─ 成本优化 (Optimization)	
7	├─ 预算管理 (Budget)	
8	├─ 预警中心 (Alerts)	
9	├─ 策略中心 (Policies)	
10	└─ 报表中心 (Reports)	

① 成本中心 (Cost Center)

这是所有计算的基础。

每个模型维护采购成本

例如：

Model	Provider	Input Cost	Output Cost	更新时间
GPT-5	OpenAI	\$1.25 / 1M	\$10 / 1M	自动同步
Claude Sonnet	Anthropic	...	...	自动同步
DeepSeek	DeepSeek	...	...	自动同步

后台支持：

- 手动修改
- 自动同步价格（后续）
- 历史价格记录

成本计算

每一次请求都会计算：

▼ 每一次请求都会计算		Plain Text
1	Prompt Token	
2		
3	+	
4		
5	Completion Token	
6		
7	=	
8		
9	真实成本	

例如：

▼		Plain Text
1	请求ID:	
2		
3	abc123	
4		
5	Input	
6		
7	6000 token	
8		
9	Output	
10		
11	2000 token	
12		
13	↓	
14		
15	采购成本	
16		
17	¥ 0.18	

所有请求都要落库。

② 定价中心 (Pricing)

很多平台没有这一层。

实际上：

采购价 ≠ 售价。

例如：

模型	成本	售价
GPT	¥0.10	¥0.15
Claude	¥0.08	¥0.13
DeepSeek	¥0.02	¥0.08

后台应该支持：

定价方式

- 固定价格
- 成本 × 倍数

例如：

▼

Plain Text |

1

售价 = 成本 × 1.5

或者：

▼

Plain Text |

1

最低利润率

2

3

30%

系统自动计算售价。

③ 利润分析 (Profit Analytics)

这个页面应该是老板最喜欢看的。

例如：

今日利润

▼

Plain Text |

1	收入
2	
3	¥ 1024
4	
5	成本
6	
7	¥ 712
8	
9	利润
10	
11	¥ 312
12	
13	毛利率
14	
15	30%

按模型分析

模型	收入	成本	利润	毛利率
GPT	¥300	¥260	¥40	13%
Claude	¥500	¥350	¥150	30%
DeepSeek	¥400	¥80	¥320	80%

你会发现：

有些模型虽然调用很多，但不赚钱。

④ 成本优化 (Optimization)

这是 Cost Engine 最有价值的地方。

例如：

后台给出建议：

	▼	Plain Text
1	最近7天：	
2		
3	GPT用于普通聊天：	
4		
5	12000次	
6		
7	建议：	
8		
9	改用DeepSeek	
10		
11	预计：	
12		
13	每月节省：	
14		
15	¥ 3800	

或者：

	▼	Plain Text
1	Claude用于摘要：	
2		
3	建议：	
4		
5	Qwen Max	
6		
7	预计：	
8		
9	节省28%	

这才是真正的智能平台。

⑤ Budget（预算管理）

针对企业客户。

例如：

某公司：

▼ Plain Text	
1	预算：
2	
3	10000元/月

已经用了：

▼ Plain Text	
1	8200

后台：

显示：

82%。

超过：

90%

提醒。

超过：

100%

自动：

限制。

甚至：

部门预算：

▼ Plain Text	
1	研发：
2	
3	5000
4	
5	运营：
6	
7	3000
8	
9	市场：
10	
11	2000

⑥ Alerts（成本预警）

实时监控异常。

例如：

今天：

▼ Plain Text	
1	OpenAI成本：
2	
3	比昨天：
4	
5	上涨300%

报警。

或者：

▼ Plain Text	
1	某API Key
2	
3	一分钟：
4	
5	调用：
6	
7	10000次

怀疑：

泄露。

报警。

又或者：

▼ Plain Text	
1	某用户
2	
3	今天：
4	
5	消费：
6	
7	¥ 5000
8	
9	超过日均：
10	
11	20倍

报警。

⑦ 策略中心 (Policies)

这是未来最重要的。

例如：

可以配置：

免费用户

▼ Plain Text	
1	优先：
2	
3	DeepSeek
4	
5	禁止：
6	
7	GPT

VIP:

▼ Plain Text	
1	允许：
2	
3	GPT
4	
5	Claude
6	
7	Gemini

夜间：

▼ Plain Text	
1	22:00以后：
2	
3	优先便宜模型

高峰：

▼ Plain Text	
1	QPS>500
2	
3	优先：
4	
5	最快Provider

利润率：

		▼	Plain Text
1	低于20%		
2			
3	禁止：		
4			
5	使用		

甚至：

		▼	Plain Text
1	Prompt		
2			
3	长度		
4			
5	>50000 token		
6			
7	↓		
8			
9	自动：		
10			
11	Claude		

这就是：

Policy Engine。

⑧ 报表中心 (Reports)

运营最喜欢。

例如：

日报：

▼		Plain Text
1	收入	
2		
3	成本	
4		
5	利润	
6		
7	调用次数	
8		
9	Top10用户	
10		
11	Top10模型	

周报：

▼		Plain Text
1	新增用户	
2		
3	API收入	
4		
5	利润变化	
6		
7	成本变化	
8		
9	模型排行	

月报：

▼		Plain Text
1	利润趋势	
2		
3	ROI	
4		
5	成本占比	
6		
7	客户排行	

可以：

导出：

Excel。

ROI分析

例如：

企业：

		Plain Text
1	A公司：	
2		
3	收入：	
4		
5	5000	
6		
7	成本：	
8		
9	4200	
10		
11	利润：	
12		
13	800	

B公司：

		Plain Text
1	收入：	
2		
3	2000	
4		
5	成本：	
6		
7	500	
8		
9	利润：	
10		
11	1500	

这样：

运营：

知道：

哪些客户值得重点服务。

模型推荐

后台：

自动分析：

	▼	Plain Text
1	最近：	
2		
3	GPT	
4		
5	很多请求：	
6		
7	其实：	
8		
9	DeepSeek即可。	
10		
11	预计：	
12		
13	节省：	
14		
15	43%	

点击：

一键：

修改：

Router。

Cost Engine 页面结构建议

▼

Plain Text

1	Cost Engine
2	
3	└─ Dashboard
4	
5	└─ Pricing
6	
7	└─ Procurement Cost
8	
9	└─ Margin Analysis
10	
11	└─ Budget
12	
13	└─ Optimization
14	
15	└─ Alerts
16	
17	└─ Policies
18	
19	└─ Reports

MVP 阶段哪些必须做？

不要一开始全部开发。

建议分阶段：

功能	MVP	V2	V3
成本计算	✓		
售价配置	✓		
利润分析	✓		
成本趋势图	✓		
预算管理		✓	
成本预警		✓	
自动优化建议		✓	

自动调价			✓
智能模型推荐			✓
AI 自动策略生成			✓

增加 Policy Engine 请求时间处理方案

只要架构设计正确，Policy Engine 的决策时间应该控制在 1~5ms，而真正耗时的是大模型推理（通常几百毫秒到几十秒）

一次 AI 请求的耗时

假设用户调用 Claude：

▼		Plain Text
1	用户	
2	↓	
3		
4	网络	
5	20ms	
6		
7	↓	
8		
9	Gateway	
10	5ms	
11		
12	↓	
13		
14	Policy Engine	
15	2ms	
16		
17	↓	
18		
19	Router	
20	2ms	
21		
22	↓	
23		
24	Anthropic API	
25	1500ms	
26		
27	↓	
28		
29	返回	
30		
31	↓	
32		
33	Gateway	
34	2ms	

整个请求：

▼		Plain Text
1	≈1530ms	

其中：

模块	耗时
----	----

Policy Engine	2ms
Router	2ms
数据库	0ms（不要查）
Claude推理	1500ms

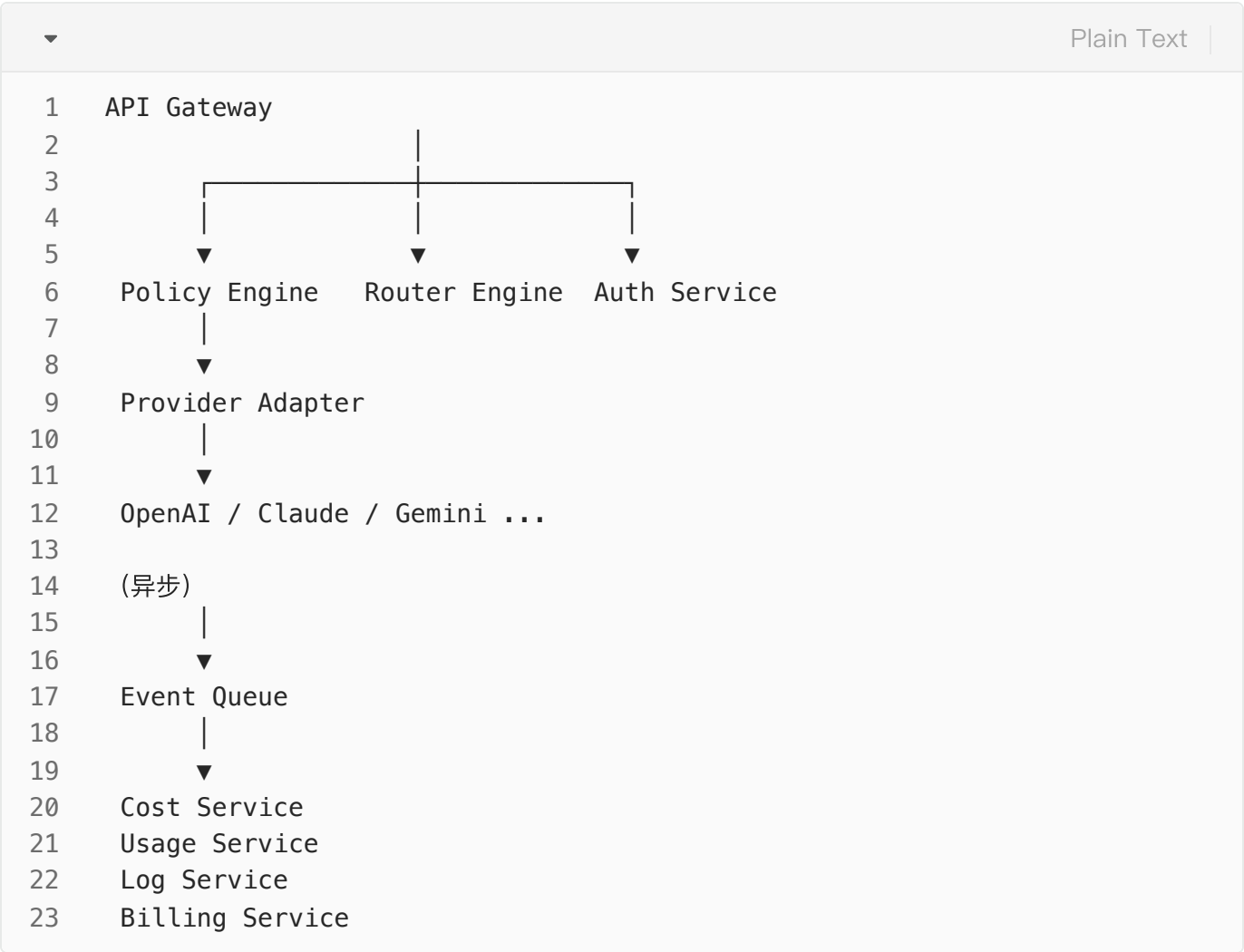
所以：

真正慢的是 Claude。

不是你的 Gateway。

对你整个项目最大的架构建议

建议把它拆成 5 个独立服务：



这样有几个好处：

- 1. 主链路极快（通常增加不到 5ms）。
- 2. 日志、成本、统计不会拖慢请求。
- 3. 后续增加企业功能（预算、审计、计费）不用修改 Gateway 核心。
- 4. 以后用户从 100 增长到 10 万，扩展也比较容易。

如果我是这个产品的架构负责人，我会把目标定为：

- API Gateway 主链路：<10ms（不含模型推理时间）
- Policy Engine：<2ms
- Router：<2ms
- 99% 的请求不访问数据库
- 所有统计、成本、日志全部异步化

这样你的产品既有企业级能力，又不会因为策略越来越复杂而拖慢响应速度。

五、Roadmap

Phase	时间	产品目标	核心功能	技术重点	商业目标	是否上线
Phase 0	2 周	项目初始化	产品定位、域名、服务器、Docker、CI/CD、数据库、Redis、Cloudflare	开发环境、测试环境、生产环境	开始开发	✗
Phase 1	4~6 周	AI Gateway MVP	登录注册、API Key、Provider、Models、Gateway、Router、Dashboard、Admin	OpenAI API 兼容、Provider Adapter	第一批种子用户	✓

Phase 2	4 周	Policy Platform	Policy Engine、Pricing、Quota、Usage、Cost、Alert、Budget	策略缓存、规则引擎、异步事件	企业开始接入	✓
Phase 3	4~6 周	Developer Platform	Playground、API Docs、SDK、Webhook、Prompt Library、Workflow(Beta)	SDK、OpenAPI、开发者体验	开发者生态	✓
Phase 4	6 周	Skill Marketplace	Skill管理、Skill发布、Skill安装、Skill收费、版本管理	Marketplace、权限系统	Skill收入	✓
Phase 5	8 周	Agent Runtime	Agent、Memory、Tool Calling、Knowledge Base、Workflow、多 Agent	MCP、RAG、任务编排	企业AI项目	✓
Phase 6	持续迭代	Enterprise AI Platform	企业权限、SSO、组织管理、私有部署、计费、审计、监控	Kubernetes、高可用、私有化	企业客户	✓

每个 Phase 对应需要开发的模块

模块	P0	P1	P2	P3	P4	P5	P6
登录注册	✓						
用户中心	✓						
API Keys		✓					
Provider 管理		✓					
Model 管理		✓					
AI Gateway		✓					

Router		✓					
Dashboard		✓					
Logs		✓					
Policy Engine			✓				
Pricing			✓				
Quota			✓				
Cost Engine			✓				
Usage Analytics			✓				
Budget			✓				
Alerts			✓				
API Playground				✓			
API Docs				✓			
SDK				✓			
Prompt Library				✓			
Workflow (基础)				✓			
Skill 管理					✓		
Skill Marketplace					✓		
Skill 收费					✓		
Agent Runtime						✓	
Memory						✓	
Tool Calling						✓	

MCP						✓	
Knowledge Base						✓	
企业组织							✓
企业权限							✓
SSO							✓
审计日志							✓
私有化部署							✓
多租户							✓

每个阶段对应需要采购的资源

Phase	服务器	域名	第三方服务	AI API
P0	新加坡测试 + 新加坡生产	2 个域名（官网、API）	Cloudflare、GitHub、Cursor	OpenAI、Claude、Gemini、DeepSeek
P1	保持现有	无新增	PostgreSQL、Redis	增加 Qwen、GLM
P2	保持现有	无新增	邮件、短信（可选）	完善模型供应商
P3	可增加对象存储	无新增	API 文档、监控	无
P4	根据 Skill 增长扩容	无新增	支付系统	无
P5	如负载增长可升级为 8C16G	无新增	向量数据库（可选）	无
P6	Kubernetes 或多节点部署	企业专属域名（按需）	企业级监控、日志、CI/CD	企业自定义模型